

Software Supply Chain

What we found, what it would cost, and what we would do

11 August 2026 · Internal

Before anything else

Nothing has happened to us.

- We went looking for evidence of an attack
- We did not find any
- This is about the doors, not a burglary

SPEAKING NOTES

Say this first and say it plainly. Any room handed a security deck assumes there has been a breach. If you do not kill that assumption in the first thirty seconds, everything else in this deck gets heard as damage control.

We checked what our software is built from, what our automated systems have been doing, and what our laptops and servers have been running. Nothing showed a break-in.

How software actually gets built

We do not write it all ourselves.

- Products are **assembled** from thousands of free components
- Published by strangers, downloaded automatically
- Like buying screws and motors instead of making them

This is normal. It is how the whole industry works.

SPEAKING NOTES

The audience needs this before anything else lands. Nobody sits down and writes a modern application from nothing — they assemble it from parts. Our automated build systems pull those parts down constantly, with no person watching.

Do not apologize for this. It is not a bad practice. It is the practice.

...which is why attackers now aim there

1. Take over the account of someone who publishes a popular component
2. Publish a new version with hostile code inside
3. Every company that builds software that week picks it up

Nobody clicked anything. Nobody made a mistake.

SPEAKING NOTES

The component was trusted last week, so it was trusted this week. That is the whole trick.

This has been happening across the industry at scale through 2026. The question this briefing answers is not “did we get hit” — we did not — it is “how well would we hold up.”

Five problems, one chain

- We **run** what suppliers send us, unread
- We hand our build systems **every key we own**
- Nine of our processes call an **address anyone can claim**
- We track **moving targets** instead of fixed ones
- Keys **never expire**, and there is no one place to say no

SPEAKING NOTES

Emphasize: these are five separate problems, and they are only serious *together*. They line up into a single chain. Each of the next few slides is one link.

1. We hand over the whole keyring

A contractor comes to fix the supply closet. We give them the master key to **every door in the building**.

Places this happens	3,506
Shared components responsible	18
Through one single component	1,904
Projects reached, that deploy to production	37

SPEAKING NOTES

The concentration is the good news. 1,904 of 3,506 go through one component, and all 3,506 go through just 18, so this is a handful of things to fix rather than thousands.

If asked which component: terraform-setup-composite-action, on tags v2 (1,904 references) and v1 (18). The others are secrets-to-tfvars-file-creator@v1 and Firenze/secrets-to-env.

If asked how, precisely: three different routes, three different fixes — an expression called `toJson(secrets)` handed to a build step (1,929 references), a setting called `secrets: inherit` on a shared process (1,363), and two steps that print the whole set into the build log's shell to read the names off it (214).

If asked why it was built this way: it was convenient, and it worked. Nobody did anything reckless. It was never revisited.

Full named list: appendix, “Problem 1a / 1b / 1c”.

2. We run supplier code on sight

Components arrive with a note attached: *“before you use me, run this.”*

We run it. Automatically. Before anyone reads it.

- **94** projects run whatever arrives
- **5** refuse

SPEAKING NOTES

This is the front door of the attack — where these campaigns begin, every time.

There is a single setting that says “download the component, but do not run its instructions.” Almost nothing of ours sets it. That is the cheapest fix in the entire deck.

If asked who already does it right: `sn-identity`, `snint-ds-e2e-testcases`, `snint-marketing-campaign-e2e-testcases` and `snip-auth-service-e2e-testcases` set it in every build; `chads-github-actions-playground` does it the better way, with one file at the top of the repository that covers every build including a developer’s laptop. The change is already written and reviewed inside our own estate — the other 94 copy it.

If asked where the change goes: either the build file at `.github/workflows/`, or a one-line file called `.npmmrc` at the top of the project. The second is preferred.

Full list of all 94: appendix, “Problem 2”.

3. Nine open doors

Nine automated processes are asking for instructions from an address where **nothing lives any more.**

- The address is **vacant, not sealed**
- Anyone inside the company can move in
- Whatever they supply gets handed the keyring

No outside attacker needed.

SPEAKING NOTES

Right now these nine are simply broken. That is the mild version.

The serious version: the addresses are not secret — anyone can read our own project files and see which ones we are calling. Somebody inside with ordinary access could claim one.

The names, if asked: `init-i`, `use-environment`, `add-copilot-blocker`, `feat/extra-node-ca-cert`, `update-to-node-24-actions`, `multiple-primary-image-versions`, `update-provider-inputs`, `dependabot-fixes`. They were deleted; the repositories that held them still exist.

Who is calling them: most are sandbox and proof-of-concept projects. The three that are not — and so the ones that matter — are `snippets-mft-ops`, `dot-env-to-env-var-action` and `eslint-config-azure-integrations`.

Both halves of the fix: repoint them, *and* reserve the names so they cannot be re-created. Repointing alone leaves the door shut but unlocked.

This is the sharpest thing we found, and it is hours of work. Appendix, “Problem 3”.

4. We track moving targets

We ask for “the current version” rather than “exactly the one we checked.”

Roughly 1 in 10 of our references name a fixed version. The other nine can change under us overnight.

SPEAKING NOTES

No approval, no change request, no notification. What runs tomorrow can differ from what ran today.

Worth noting this is also a reliability problem, not only a security one — see the benefits slide.

The biggest ones, if asked: `github-env-vars-action` (233 references), `setup-terraform` (162), `action-dotenv-to-setenv` (111), `deployment-action` (92). Those four alone are where fixing this buys the most.

Related, and worth naming: four of our builds download code from the internet and run it immediately without checking it — in `sleep-number-claude-code-plugins`, `sdna-new-databricks` and `sdp-databricks-pytest-poc`. Each is deliberate, but it is the same primitive the attack relies on. Appendix, “Problem 4”.

What happens if we do not fix it

- **A second of access becomes every credential we own**
- Cannot prove what was taken → must assume **everything**
- Rotate every key, redeploy, under pressure, work stops
- Hard to answer *“what did they take, and where did they go?”*

There is **no small version** of this incident.

SPEAKING NOTES

Be concrete rather than dramatic. Nobody is claiming the company falls over.

The expensive part is the clean-up, and it is triggered by an event that might last seconds. That is the asymmetry worth funding against.

Worth doing even if nobody attacks us

- **Nine broken automations** get repaired
- Builds **stop changing underneath us** — fewer mystery failures
- A real **list of what our software is made of** — what auditors ask for
- **Less exposure to someone else's outage**
- Incidents get **shorter**, because keys expire on their own

SPEAKING NOTES

This is the slide that gets the work funded. Several of these fixes pay for themselves with no attacker involved at all.

The inventory point lands especially well with anyone who has been asked by a customer or an auditor what our software contains. Today that question takes a project to answer.

Now — weeks, not quarters, nothing to buy

A. Close the nine open doors — hours Repoint them, and lock the names so nobody can claim them

B. Stop running supplier code on sight — one line per project 94 projects, cheapest protection here

C. Stop handing over the whole keyring — a few weeks A small number of shared components, not thousands of projects

SPEAKING NOTES

Order matters. A first — it is the sharpest risk and the least effort.

On A: both halves. Repointing without locking the name leaves the door shut but unlocked.

On C: it sounds enormous and it is not, because the exposure is concentrated — 18 shared components, and one of them accounts for 1,904 of the 3,506 places.

Which teams this lands on: the platform team that owns `terraform-setup-composite-action` carries most of C. B is spread across 94 project teams, but each one is a single line — it batches. A is a handful of central workflow repositories.

Every name behind every number is in the appendix, organized by problem, generated from the scan data rather than typed. If anyone wants their own repository's status, it is in there.

Next — a quarter each

D. Badges instead of keys Short-lived passes issued per job, expiring in minutes. Nothing durable left to steal.

E. One front door for outside software One place to block a bad component everywhere at once, and a short waiting period before new versions can be used.

SPEAKING NOTES

D is what makes C permanent rather than a setting somebody can quietly undo later. We already own this capability and are barely using it.

Neither of these is a purchase. Both are configuration of things already licensed.

We already own the front door

Almost nothing goes through it. **The mechanism is proven inside our own walls** — it was simply never pointed at this category of software.

Through our internal supply point

Microsoft-ecosystem components	523
The components in question here	0

SPEAKING NOTES

This is the strongest slide in the deck for anyone worried about cost. We are not proposing to build something. It exists, it is connected, and one category of software already flows through it successfully.

The feeds, if asked: five of them, across two Azure DevOps organizations — `sn-tim`, `sn-tim-packages`, `sn-tim-invision-packages`, `SleepNumberIndigo`, and `k8s-manifests`. Three already have the connection to the public catalog configured and unused.

One caveat to state rather than skip: we could not read `k8s-manifests` — we lack the permission. A request naming exactly what is needed is filed. So the zero on this slide covers four feeds of five, not five of five. Appendix, “Fix E”.

Why a waiting period works

The catalog cleans itself up — but only after a few days, and **our builds are fast enough to catch them in that gap.**

Bad versions published in this campaign	2,208
Already pulled from the catalog when we checked	2,097
Still out there	111

SPEAKING NOTES

Around 95% withdrawn. Make a version sit unused for a few days and most of these are gone before we could ever have touched them.

The remaining 111 are exactly why we need the block list as well as the delay. Do not oversell the waiting period as a complete answer.

What we are not claiming

- We did **not** check everything — **193** projects unread
- **Not checked is not the same as clean**
- Personal laptops are largely **outside** this
- One internal supply point we **could not read**

“No evidence of a break-in” is not “no break-in.”

SPEAKING NOTES

Include this slide. Do not cut it for time.

A briefing that overstates its confidence is worth less than one that does not, and this audience has no way to catch us at it. Saying where we could not look is what makes the rest credible.

Most real victims of these campaigns were hit on a developer's own laptop, not a build server. Our visibility there is thinner.

In one minute

We build our products partly from free components downloaded automatically from the internet. Attackers have started poisoning those at the source, and it is working across the industry.

We checked. It has not reached us.

If it did, three things would turn a small event into a large one — and all three cost **weeks, not quarters**, with **nothing to purchase**.

SPEAKING NOTES

If you get five minutes instead of thirty, this is the slide. Open on it, and take questions.

The three: we run supplier code on sight, we hand over every key instead of one, and nine processes call an address anyone inside could claim.

Appendix — the specific names

How to read this appendix

The briefing deliberately uses counts rather than names — the names crowd out the argument for a reader who does not own any of these repositories.

This appendix is the other half: **every count in the briefing, resolved to the things a team would actually open and change.** It is generated from the same collector artifacts the technical report reads, not typed by hand, so it cannot drift from them.

Anything listed here is something we read directly. Where a population could not be read at all, it is named at the end as a gap rather than left out.

Problem 1a — build components handed the whole keyring

5 components are passed to `JSON(secrets)` — the complete secrets object — across **1,929** pipeline references. **4** of them are tracked by a moving name rather than an exact version, covering **1,924** of those references. The one pinned to a long commit reference is safer against tampering but still receives everything.

Build component handed every secret	Pipeline refs	Shared workflows	Deploys
terraform-setup-composite-action@v2	1,904	40	yes
terraform-setup-composite-action@v1	18	8	yes
Firenza/secrets-to-env@7da604dcd013382fa0b2c9b6aeb33400f10bf322	5	1	—
Firenza/secrets-to-env@v1.3.0	1	1	—
secrets-to-tfvars-file-creator@v1	1	1	yes

Problem 1b — shared processes handed the whole keyring (1 of 2)

A second mechanism, and a separate fix: **11** shared processes are called with `secrets: inherit`, which passes everything the caller holds without naming any of it. **1,363** pipeline references. Almost all of these deploy.

Shared process handed every secret	Pipeline refs	Callers	Deploys
terraform-format-gha-workflow › terraform_format.yaml@v1	278	12	yes
terraform-security-gha-workflow › terraform_security.yaml@v1	275	11	yes
check-destroy-workflow › check_destroy.yaml@v1	252	10	yes
semantic-release-deployment-workflow › semantic_release_deployment.yaml@v1	246	6	yes
promote-release-and-deployment-workflow › promote_release_and_deployment.yaml@v1	245	9	yes
terraform-testing-workflows › test_terraform.yml@v1	42	2	—

Problem 1b — shared processes handed the whole keyring (2 of 2)

Shared process handed every secret	Pipeline refs	Callers	Deploys
terraform-testing-workflows > test_terraform.yml@v2	10	2	yes
cicd-workflows > promote_release_and_deployment.yml@v1	6	4	yes
audit-github-scanner-workflow > scan.yml@v1	3	1	—
ado-work-item-check > check_commit.yml@v1	3	2	yes
ado-work-item-update > update_work_item.yml@v1	3	2	yes

Problem 1c — steps that print the keyring into the shell

2 steps expand the whole secrets object into the runner's own command line in order to read the *names* off it — **214** pipeline references. Reading names never requires handling values, so this one is a rewrite rather than a narrowing.

Step that expands secrets into the build shell	Pipeline refs	Deploys
verify-secrets-dev:Get secret JSON keys	107	—
verify-secrets-all-env:Get secret JSON keys	107	—

Problem 1 — where the change lands

Three mechanisms, three different edits. All of them live in `.github/workflows/<name>.yaml` in the shared repository, not in the consumers. For 1a and 1b there is a second step: pin the target to a commit reference instead of `@v1` / `@v2`, and turn on tag protection in the target's own repository so the name cannot be moved.

Mechanism	What to look for	What to replace it with
1a	<code>\$</code> <code>{{ toJSON(secrets)</code> <code>}}</code> in a <code>with: block</code>	the specific secrets that step reads, named one per line
1b	<code>secrets: inherit</code> under a <code>uses: call</code>	a <code>secrets: block</code> naming only what the called process needs
1c	<code>toJSON(secrets)</code> inside a <code>run: script</code>	an approach that lists secret names without expanding their values

Problem 1 — the shared workflows that pass it onward (largest of 46) (1 of 2)

46 definitions pass the whole secrets context to whatever called them, reaching **1,823** consumer references. **37** of them deploy. The full 46 are in `exports/hunt/reusable_workflow_targets.json`; these are the largest.

Shared workflow	Ref	Consumers	Deploys
terraform-delete-gha-workflow › delete_resource.yaml	v2	246	yes
terraform-unlock-gha-workflow › unlock_state.yaml	v2	246	yes
terraform-rename-gha-workflow › rename_resource.yaml	v2	246	yes
terraform-import-gha-workflow › import_resource.yaml	v2	246	yes
verify-secrets-gha-workflow › verify_secrets.yaml	v1	107	—
terraform-infra-ci-gha-workflow › terraform_ci.yaml	v2	103	yes
terraform-infra-cd-gha-workflow › terraform_cd.yaml	v2	96	yes
pr-comment-deploy-managed-api-gha-workflow › pr_comment_deploy.yaml	v2	67	yes

Problem 1 — the shared workflows that pass it onward (largest of 46) (2 of 2)

Shared workflow	Ref	Consumers	Deploys
managed-api-ci-gha-workflow > managed-api-ci.yaml	v2	67	yes
managed-api-cd-gha-workflow > managed-api-cd.yaml	v2	66	yes
pr-comment-deploy-function-app-gha-workflow > pr_comment_deploy.yaml	v2	60	yes
node-function-app-ci-gha-workflow > node-function-app-ci.yaml	v2	55	yes
node-function-app-cd-gha-workflow > node-function-app-cd.yaml	v3	54	yes
terraform-destroy-gha-workflow > destroy.yaml	v1	39	yes
terraform-module-ci-gha-workflow > terraform_module_ci.yaml	v1	21	—
node-multi-deployment-function-app-ci-gha-workflow > node-multi-deployment-function-app-ci.yaml	v2	13	yes

Problem 1 — the remaining shared workflows (1 of 2)

The other 30 definitions, each passing the whole secrets context onward. Smaller consumer counts, same mechanism.

node-multi-deployment-function-app-cd-gha-workflow@v4	pr-comment-deploy-multi-deployment-function-app-gha-workflow@v2
static-web-app-cd-gha-workflow@v1	java-maven-ci-gha-workflow@v2
logic-app-ci-gha-workflow@v1	terraform-module-ci-gha-workflow@v2
logic-app-cd-gha-workflow@v1	pr-comment-deploy-logic-app-gha-workflow@v1
container-app-ci-gha-workflow@v1	container-app-cd-gha-workflow@v1
terraform-delete-gha-workflow@v1	terraform-import-gha-workflow@v1
terraform-rename-gha-workflow@v1	terraform-unlock-gha-workflow@v1
xslt-mapping-cd-gha-workflow@v1	xslt-mapping-ci-gha-workflow@v1

Problem 1 — the remaining shared workflows (2 of 2)

terraform-infra-cd-gha-workflow@v1	terraform-infra-ci-gha-workflow@v1
web-app-cd-gha-workflow@v2	web-app-ci-gha-workflow@v2
java-maven-ci-gha-workflow@secrets-to-env-vars	logic-app-cd-gha-workflow@add-copilot-agent-fix
node-function-app-cd-gha-workflow@new-test	node-function-app-cd-gha-workflow@v2
node-function-app-ci-gha-workflow@v1	pr-comment-deploy-function-app-gha-workflow@main
pr-comment-deploy-infra-gha-workflow@v1	pr-comment-deploy-managed-api-gha-workflow@test-update4
python-function-app-ci-gha-workflow@initial-creation	terraform-destroy-gha-workflow@initial-code

Problem 2 — projects that run supplier code on sight (1 of 5)

All **94** of them. Each needs `--ignore-scripts` on its install command, or `ignore-scripts=true` in an `.npmrc` committed at the repository root.

<code>SleepNumber/.com</code>	<code>SleepNumber/actions-sandbox</code>
<code>SleepNumber/sleepcms</code>	<code>SleepNumber/sndotcom</code>
<code>SleepNumber/vtex-boilerplate-app</code>	<code>SleepNumberInc/App-Factory-Trial</code>
<code>SleepNumberInc/Azure-Function-Node-POC</code>	<code>SleepNumberInc/Azure-Integrations-Managed-APIs</code>
<code>SleepNumberInc/AzureIntegration</code>	<code>SleepNumberInc/ProductCatalogLogicAppsTests</code>
<code>SleepNumberInc/SCM-Supplier-Stock-Automation-fna</code>	<code>SleepNumberInc/Workforce-Mgmt-Managed-APIs</code>
<code>SleepNumberInc/authorize</code>	<code>SleepNumberInc/aws-waf-blocked-ip-ruleset-cron</code>
<code>SleepNumberInc/cognito-user-pool-maintenance</code>	<code>SleepNumberInc/common-logging-test</code>
<code>SleepNumberInc/cryptex</code>	<code>SleepNumberInc/custom-logger</code>
<code>SleepNumberInc/dev-js-config</code>	<code>SleepNumberInc/devops-github-org-webhook-processor</code>

Problem 2 — projects that run supplier code on sight (2 of 5)

<code>SleepNumberInc/docker-swarm-service-balancer</code>	<code>SleepNumberInc/dot-env-to-env-var-action</code>
<code>SleepNumberInc/ebs-order-integration</code>	<code>SleepNumberInc/exp-ingestion-api</code>
<code>SleepNumberInc/exp-sbn-api</code>	<code>SleepNumberInc/expired-secrets-terminator</code>
<code>SleepNumberInc/gcc-creative-studio</code>	<code>SleepNumberInc/gh-token-rate-limit-monitor</code>
<code>SleepNumberInc/github-runner-group-usage-monitor</code>	<code>SleepNumberInc/github-token-rate-limit-monitor</code>
<code>SleepNumberInc/java-linter-output-formatter</code>	<code>SleepNumberInc/javascript-github-action-template</code>
<code>SleepNumberInc/job-data-updater</code>	<code>SleepNumberInc/node-cron-docker</code>
<code>SleepNumberInc/nodejs-microservice-template</code>	<code>SleepNumberInc/nodejs-microservice-utils</code>
<code>SleepNumberInc/npm-package-cd-workflow</code>	<code>SleepNumberInc/npm-package-ci-workflow</code>
<code>SleepNumberInc/opentelemetry-js-agent</code>	<code>SleepNumberInc/opentelemetry-plugin-nestjs-rabbitmq</code>

Problem 2 — projects that run supplier code on sight (3 of 5)

SleepNumberInc/order-status-poc-api	SleepNumberInc/proc-product-catalog
SleepNumberInc/product-catalog-e2e	SleepNumberInc/product-exchange-backend
SleepNumberInc/product-exchange-frontend	SleepNumberInc/pulse-sandbox
SleepNumberInc/scm-poc-fna	SleepNumberInc/scmtstfuncapp
SleepNumberInc/sec-diligence	SleepNumberInc/secret-server-secret-retriever
SleepNumberInc/secrets-to-tfvars-file-creator	SleepNumberInc/servicenow-devops-change
SleepNumberInc/sharedsvc-notification-webhook-fna	SleepNumberInc/sn-identity-auth-test-ui
SleepNumberInc/sn-vare-product-exchange-backend	SleepNumberInc/sn-vare-product-exchange-frontend
SleepNumberInc/snint-azure-gop-fna	SleepNumberInc/snint-contact-merges-siebel-publisher-fna

Problem 2 — projects that run supplier code on sight (4 of 5)

SleepNumberInc/snint-customer-lookup-process-api-fna	SleepNumberInc/snint-customer-orders
SleepNumberInc/snint-ds-k6-load-tests	SleepNumberInc/snint-ebs-inventory-pollers-fna
SleepNumberInc/snint-employees-api	SleepNumberInc/snint-employees-d365-consumer-fna
SleepNumberInc/snint-employees-mapper-api	SleepNumberInc/snint-fn-test-sftp-onpremise-dev
SleepNumberInc/snint-incidents-d365-consumer-fna	SleepNumberInc/snint-incidents-d365-customer-lookup-fna
SleepNumberInc/snint-incidents-digital-publisher-apim	SleepNumberInc/snint-incidents-digital-publisher-fna
SleepNumberInc/snint-invision-mappings	SleepNumberInc/snint-logs-poc-fna
SleepNumberInc/snint-logs-poc2-fna	SleepNumberInc/snint-logs-without-ai-poc-fna
SleepNumberInc/snint-pim-api-fna	SleepNumberInc/snint-pim-conn-fna

Problem 2 — projects that run supplier code on sight (5 of 5)

SleepNumberInc/snint-products-fna	SleepNumberInc/snint-stores-siebel-publisher-fna
SleepNumberInc/snip-cli	SleepNumberInc/snip-e2e-ui-poc
SleepNumberInc/snmspsal-ai-sales-trainer-backend-fna	SleepNumberInc/snoint-azure-integrations-order-fna
SleepNumberInc/snoint-siebel-gop-api-fna	SleepNumberInc/sys-product
SleepNumberInc/sys-rule	SleepNumberInc/terratest-json-output-fixer
SleepNumberInc/test-microservice	SleepNumberInc/test-ms-template
SleepNumberInc/test-ms-template2	SleepNumberInc/test-results-file-to-qtest-action
sleepnumberlabs/poc-john-merrill-testing	sleepnumberlabs/qa-sn-playwright-automation
sleepnumberlabs/web-consumer-sleepiq	sleepnumberlabs/web-support-portal

Problem 2 — the projects that already refuse

5 repositories are already protected, covering **6** workflows. They are the pattern to copy — the change is already written and reviewed inside our own estate.

Repository	How it is protected
SleepNumberInc/chads-github-actions-playground	.npmrc config
SleepNumberInc/sn-identity	every installing workflow
SleepNumberInc/snint-ds-e2e-testcases	every installing workflow
SleepNumberInc/snint-marketing-campaign-e2e-testcases	every installing workflow
SleepNumberInc/snip-auth-service-e2e-testcases	every installing workflow

Problem 2 — where the change lands

- **In a workflow:** `.github/workflows/<name>.yaml` — add `--ignore-scripts` to the `npm ci / npm install / yarn / pnpm` line
- **Or, better, once per repository:** create `.npmrc` at the repository root containing `ignore-scripts=true`, which covers every install including a developer's laptop

The second form is preferred: it is one file, it is reviewable, and it protects the surface the first form does not.

Problem 3 — the vacant addresses

9 references across 12 consumer calls point at these branch names in central workflow repositories. The repositories exist; the branches were verified deleted. Two names carry more than one reference, which is why nine references map to eight distinct names.

Branch name being called	Status
initi	deleted — claimable
use-environment	deleted — claimable
add-copilot-blocker	deleted — claimable
feat/extra-node-ca-cert	deleted — claimable
update-to-node-24-actions	deleted — claimable
multipe-primary-image-versions	deleted — claimable
update-provider-inputs	deleted — claimable
dependabot-fixes	deleted — claimable

Problem 3 — who is calling them, and what to do

Consumers that are not sandboxes — these break today and are the escalation path:

- `SleepNumberInc/snip-iics-mft-ops`
- `SleepNumberInc/dot-env-to-env-var-action`
- `SleepNumberInc/eslint-config-azure-integrations`

The rest are sandbox and proof-of-concept repositories (`cldsvcs-test-*`, `devops-sandbox-*`, `ss_gh_poc`, `chads-github-actions-playground`).

Two more are ordinary breakage rather than risk: `snip-iics-mft-ops` calls `iics_cd_workflow.yaml@main` where the file is now `iics_cd.yaml`, and two repos call `pr_lint.yaml@v1` where the file is `pr_lint.yaml`.

The fix has two halves, and both are required. Repoint the `uses:` line at a commit SHA on a branch that exists — then add a branch protection rule in the *central* repository matching each deleted name, and restrict branch creation to maintainers. Repointing alone leaves the door shut but unlocked.

Problem 4 — the most-referenced moving targets

8,115 references sit on a moving name against **960** pinned to an exact version. These twelve are where pinning buys the most.

Outside component tracked on a moving name	References
FranzDiebold/github-env-vars-action@v2.1.0	233
hashicorp/setup-terraform@v1	162
c-py/action-dotenv-to-setenv@v3	111
chnorm/deployment-action@releases/v1	92
ncipollo/release-action@v1	73
Azure/docker-login@v1	70
montudor/action-zip@v0.1.1	63
notiz-dev/github-action-json-property@release	62
rsotnychenko/deployment-status-update@0.2.0	60
jossef/action-set-json-field@v1	55
Azure/container-scan@v0	52
Firenza/secrets-to-env@v1.1.0	52

Problem 4 — workflows that download and run code without checking it

Four workflows fetch code from the internet and execute it immediately. This is deliberate in each case, but it is the same primitive the campaign relies on, so each should be pinned to a known version or replaced.

Repository	File	Occurrences
sleep-number-claude-code-plugins	.github/workflows/ ci.yaml	2
sleepnumberlabs/sdna-new-databricks	.github/workflows/ 1-sync-databricks- jobs.yml	1
sleepnumberlabs/sdp-databricks-pytest-poc	.github/workflows/ run-integration- tests.yml	1
sleepnumberlabs/sdp-databricks-pytest-poc	.github/workflows/ run-tests-on- databricks-dev.yml	1

Fix E — the front door we already own

0 relevant components flow through any of these, against **523** Microsoft-ecosystem components that do. The connection already exists on three of them.

Feed	Connected to the public catalog	Readable
sn-tim/sn-tim	yes	yes
sn-tim/sn-tim-invision-packages	no	yes
sn-tim/sn-tim-packages	yes	yes
SleepNumberIndigo/k8s-manifests	no	no
SleepNumberIndigo/SleepNumberIndigo	yes	yes

What is deliberately not listed here

- **193 projects** where the download step happens inside something we did not read. They are neither safe nor unsafe in this appendix, because we do not know.
- **265 projects** with no automated install at all, **73** of them with no automation whatsoever. Their components are only ever downloaded on a person's own laptop.
- **2 repositories** whose file listing was cut short by GitHub: SleepNumberInc/Github_PBI_Integrated_Workspaces, sleepnumberlabs/siqassess.
- **One internal feed we could not read** — SleepNumberIndigo/k8s-manifests — for permission reasons. A request naming the exact permission is in the technical report.
- **The 838 workflows that put secret values into a command line.** The collector records the first 200; listing a truncated set here would read as a complete one.

Generated by scripts/report/build_appendix.py from the collector artifacts in exports/hunt/. Re-run it rather than editing this file.